

Practical 3: Message Authentication Codes

AIM: Implement algorithms to generate and verify message authentication codes (MACs) for ensuring data integrity and authenticity.

CLI CODE:

```
import hmac
import hashlib
```

```
secret_key = input("Enter the secret key: ").encode()
message = input("Enter the message: ").encode()
```

```
mac = hmac.new(secret_key, message, hashlib.sha256).hexdigest()
```

```
print("\nGenerated MAC:", mac)
```

```
received_message = input("\nEnter the message for verification: ").encode()
```

```
received_mac = hmac.new(secret_key, received_message, hashlib.sha256).hexdigest()
```

```
print("\nVerifying Message...")
```

```
if hmac.compare_digest(mac, received_mac):
```

```
    print("Authentication Successful!")
```

```
    print("Message Integrity Verified.")
```

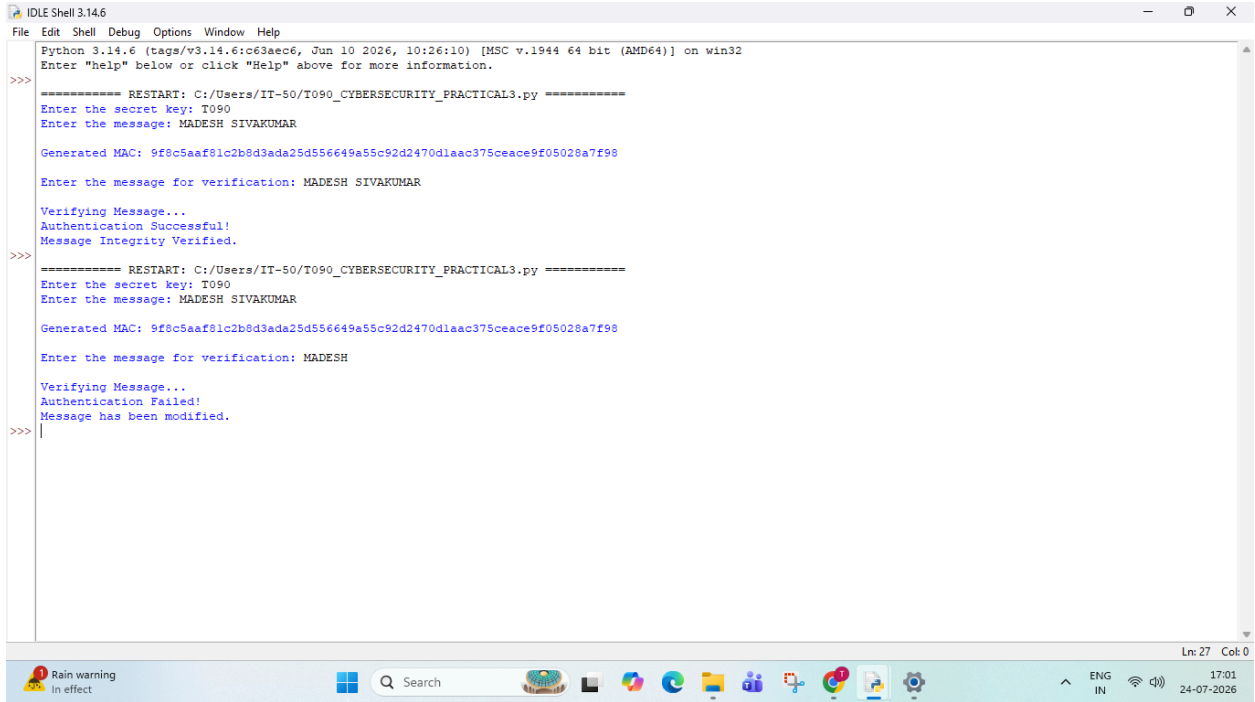
```
else:
```

```
    print("Authentication Failed!")
```

```
    print("Message has been modified.")
```

OUTPUT:

SHETH L.U.J. AND SIR M.V. COLLEGE



```
IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aee6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/IT-50/T090_CYBERSECURITY_PRACTICAL3.py =====
Enter the secret key: T090
Enter the message: MADESH SIVAKUMAR

Generated MAC: 9f8c5aaf81c2b8d3ada25d556649a55c92d2470d1aac375ceace9f05028a7f98

Enter the message for verification: MADESH SIVAKUMAR

Verifying Message...
Authentication Successful!
Message Integrity Verified.

>>>
===== RESTART: C:/Users/IT-50/T090_CYBERSECURITY_PRACTICAL3.py =====
Enter the secret key: T090
Enter the message: MADESH SIVAKUMAR

Generated MAC: 9f8c5aaf81c2b8d3ada25d556649a55c92d2470d1aac375ceace9f05028a7f98

Enter the message for verification: MADESH

Verifying Message...
Authentication Failed!
Message has been modified.

>>>
```

GUI OUTPUT:

```
import tkinter as tk
from tkinter import messagebox
import hmac
import hashlib
```

```
def generate_mac():
    key = entry_key.get().encode()
    message = entry_message.get().encode()

    if not key or not message:
        messagebox.showerror("Error", "Please enter both key and message")
        return
```

```
    mac = hmac.new(key, message, hashlib.sha256).hexdigest()
    entry_mac.delete(0, tk.END)
    entry_mac.insert(0, mac)
```

```
def verify_mac():
    key = entry_key.get().encode()
    message = entry_verify_message.get().encode()
    mac_original = entry_mac.get()
```

```
    if not key or not message or not mac_original:
        messagebox.showerror("Error", "Please fill all fields")
```

T090_MADESH SIVAKUMAR

```
    return

    new_mac = hmac.new(key, message, hashlib.sha256).hexdigest()

    if hmac.compare_digest(mac_original, new_mac):
        messagebox.showinfo("Result", "✅ Authentication Successful!\nMessage Integrity
Verified.")
    else:
        messagebox.showerror("Result", "❌ Authentication Failed!\nMessage has been
modified.")

root = tk.Tk()
root.title("MAC Generator & Verifier")
root.geometry("500x400")

tk.Label(root, text="Secret Key").pack()
entry_key = tk.Entry(root, width=50)
entry_key.pack()

tk.Label(root, text="Message (Generate MAC)").pack()
entry_message = tk.Entry(root, width=50)
entry_message.pack()

tk.Button(root, text="Generate MAC", command=generate_mac, bg="lightblue").pack(pady=5)

tk.Label(root, text="Generated MAC").pack()
entry_mac = tk.Entry(root, width=50)
entry_mac.pack()

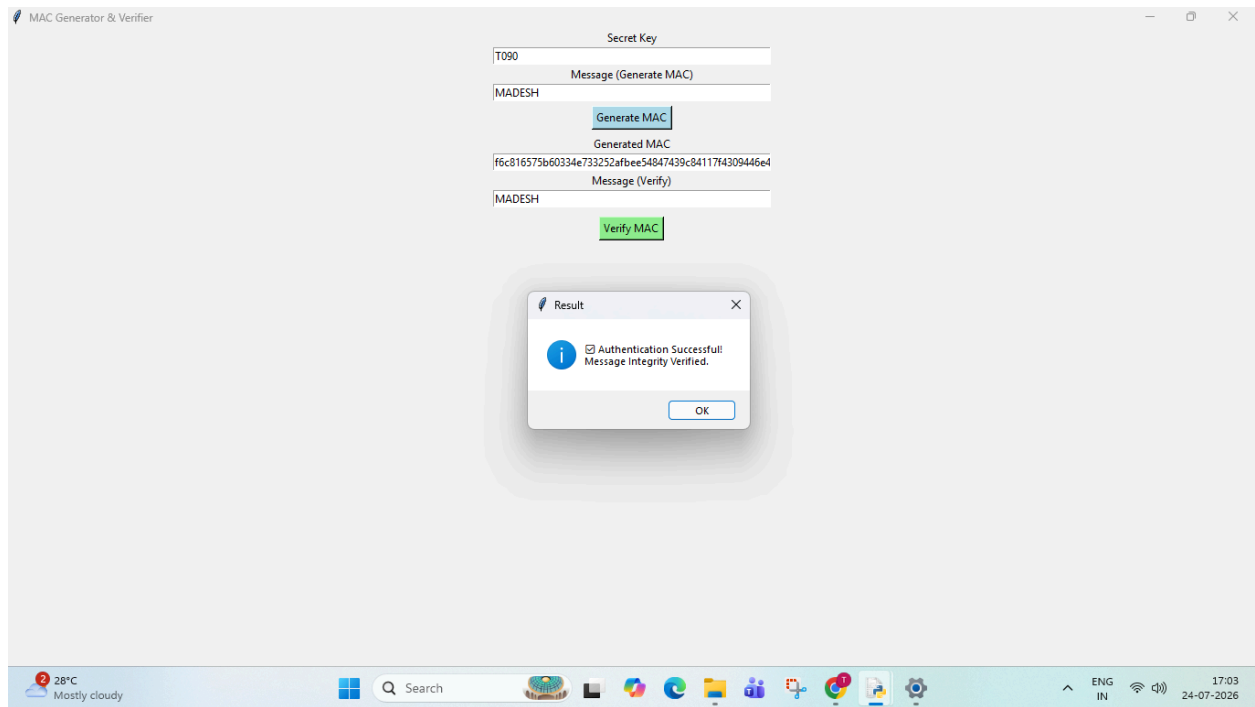
tk.Label(root, text="Message (Verify)").pack()
entry_verify_message = tk.Entry(root, width=50)
entry_verify_message.pack()

tk.Button(root, text="Verify MAC", command=verify_mac, bg="lightgreen").pack(pady=10)

root.mainloop()
```

SHETH L.U.J. AND SIR M.V. COLLEGE

OUTPUT:



T090_MADESH SIVAKUMAR